

Estimating How Much Memory a Program Uses over Time from Partial Instrumentation of Its Memory Accesses

Nafis Mustakin
University of California, Riverside

October 2026

Abstract

Knowing how much memory a program touches, and when, helps compilers, runtime systems and job schedulers decide where to place data and how many programs fit on one machine. The most direct way to measure it is to record every memory access with a binary instrumentation tool such as Pin, but recording every access slows a program down by two to three orders of magnitude. MemPrint reduces that cost by recording only a random fraction of the accesses and by learning, from small inputs that were recorded completely, how to scale the sampled result up to the true total. MemPrint predicts a single number for a whole run. In this report we extend the problem to the amount of memory in use at every point of a run, with freed memory removed, and we describe in detail every approach we tried, including those that failed, because the failures explain the design that works. We find three things. First, estimators that work from a random sample of memory accesses cannot tell memory that is used for the first time from memory that is used again, and at a sampling rate of 1 in 25 accesses the best such estimator still has 24% mean error on a larger input. Second, sampling memory locations instead of accesses removes that difficulty, because every location is chosen with the same probability no matter how often it is used. This estimator needs no training, and its mean error is 0.9% to 4.2% on PolyBench and 0.2% to 6.1% on miniVite with 1, 4 and 16 threads. Third, sampling locations still requires a check on every access, which costs $21\times$ to $36\times$ the native running time on large inputs. We therefore check accesses only during short windows of time, and between windows we track only memory allocations, the pages that the operating system reports as touched, and, for memory that the allocator reuses, the pages written since each allocation. Watching 5% of the run, the resulting estimate is within 0.2% of the truth on PolyBench, within 3.1% to 3.8% on miniVite with 65536 vertices, and within 24% on a test that reuses freed memory heavily, where the touched pages alone are 248% too high. It costs $2.8\times$ to $3.2\times$ native time on PolyBench and $13\times$ to $15\times$ on miniVite.

Contents

1	Introduction	3
2	Terms	4
3	Background	5
3.1	Dynamic binary instrumentation with Pin	5
3.2	MemPrint	5

4	Measuring the live footprint over time	6
4.1	Removing freed memory	6
4.2	Snapshots and partial recordings	7
4.3	Ordering events across threads	8
4.4	Tests	8
5	Evaluation method	8
5.1	Workloads, inputs and machine	8
5.2	Ground truth and error metrics	9
6	Estimating from sampled references	9
6.1	The α model at every snapshot	9
6.2	Estimators of unseen species	10
6.3	An estimator that uses the known sampling rate	10
6.4	A hybrid of the known-rate estimate and a learned correction	11
7	Sampling addresses	12
7.1	Method	12
7.2	Results	13
8	Windowed sampling	14
8.1	Counting time between windows	14
8.2	Switching between watching and not watching	15
8.3	Tracking live blocks	15
8.4	Page residency	16
8.5	Written pages of reused blocks	16
8.6	Chunk sampling and density	17
8.7	The estimate	18
8.8	Approaches that did not work	18
9	Evaluation of windowed sampling	18
9.1	PolyBench	19
9.2	miniVite	19
9.3	The peak of miniVite with 32768 vertices	19
9.4	Heap churn	20
9.5	Cost	21
10	Discussion and limitations	22
11	Related work	23
12	Next steps	24
13	Conclusion	25
A	Options of the Pin tool	28
B	Output files	28

1 Introduction

A program’s *memory footprint* is the amount of memory it actually uses. Many decisions depend on it. A job scheduler needs to know how much memory a job will need before it places the job on a machine. A system with two kinds of memory, for example fast DRAM and slower but larger memory, needs to know which data is used and when, so that it can keep that data in the fast memory [1, 35]. A compiler that reorganizes data needs to know which parts of an array a loop touches.

Measuring the footprint is harder than it seems. A heap profiler such as Valgrind’s Massif [36] records every call to the memory allocator, but an allocation only says that memory was reserved, not that it was used, and a program can reserve far more memory than it touches. A heap profiler also does not see the stack, global variables, or memory that libraries request directly from the operating system. The most direct measurement records every memory access with a dynamic binary instrumentation tool, which is a tool that inserts extra code into a running program. Pin [25], Valgrind [29] and DynamoRIO [6] are such tools. Recording every access slows a program down by two to three orders of magnitude, which makes the measurement impractical for large programs.

MemPrint [28] reduces that cost by recording only a random fraction of the memory accesses. It then learns, from small inputs of the same program that were recorded completely, a correction factor that scales the footprint seen in the sample up to the true footprint. MemPrint predicts one number per run: the total amount of memory touched over the whole run.

Many uses need more than one number. The amount of memory in use changes during a run, because a program allocates memory, uses it and frees it again. The largest amount in use at any one time, which we call the *peak*, decides how much memory a machine must have, and the time at which the peak occurs decides when that memory is needed. In this report we extend MemPrint to estimate the amount of memory in use at every point of a run, with freed memory removed. We record every approach we tried, including the approaches that failed, because the reasons for the failures explain the design that works.

In this work, we make the following contributions:

- We extend the MemPrint Pin tool so that it removes freed memory from its measurements, writes the measurements at regular points in time, and can stop recording partway through a run. We find and fix two errors in how the tool ordered events from different threads, which made the measured peak of a multithreaded test up to 19% too low (Section 4).
- We show that estimators which work from a random sample of memory accesses cannot reach low error at sparse sampling rates, because such a sample cannot tell new memory from memory that is used again. This holds for the MemPrint model applied at every point in time, for two estimators from ecology, and for a new estimator that uses the known sampling rate together with a learned correction (Section 6).
- We propose sampling memory locations instead of memory accesses, and show that it estimates the memory in use over time without any training (Section 7).
- We propose windowed sampling, which samples memory locations only during short windows of time, and which tracks allocations, touched pages and written pages for the whole run at low cost (Section 8 and Section 9).
- We evaluate every estimator against complete recordings on PolyBench [34], miniVite [17] and a synthetic test that reuses freed memory, and we list the limitations and the next steps (Section 10 and Section 12).

The rest of this report is organized as follows. Section 2 defines the terms we use. Section 3 describes Pin and MemPrint. Section 4 describes the changes we made to the Pin tool to measure memory over time, and Section 5 describes how we evaluate an estimate. Sections 6 to 9 present the three families of estimators in the order in which we developed them. Section 10 discusses limitations, Section 11 discusses related work, and Section 12 lists next steps. The appendices list the tool’s options and output files.

2 Terms

We define here the terms used in the rest of the report, in an order in which each definition uses only earlier ones.

Memory reference. A *memory reference* is one read or one write of memory by one machine instruction. An instruction that both reads and writes the same memory location, for example an instruction that adds a register to a value in memory, makes two references. Each reference has a *start address*, which is the address of the first byte it touches, and a *size*, which is the number of bytes it touches, for example 8 bytes for a 64-bit integer.

Footprint. Following MemPrint, the *footprint* of a set of memory references is the sum, over all distinct start addresses that the references use, of the largest size seen at each start address. For example, an 8-byte read at address A and a 4-byte write at the same address A add 8 bytes to the footprint, because only the largest size at each start address counts. However, an 8-byte read at A and an 8-byte read at $A + 4$ add 16 bytes, because they have different start addresses, even though together they cover only 12 bytes. The footprint therefore counts overlapping accesses more than once, which matters in Section 8.6.

Live footprint and peak. The *live footprint* at a point in time t is the footprint of all references made up to t , after removing every address inside a memory range that the program freed before t . When the program frees a range, the addresses in it stop counting at that moment, and if the program later uses the same addresses again, they count again. The *peak* is the largest live footprint over the whole run.

Time. We measure time in memory references executed since the start of the run, rather than in seconds. Instrumentation slows different parts of a program down by different factors, so seconds measured under instrumentation do not correspond to seconds in a run without instrumentation, whereas the number of references executed is the same.

Snapshot and timeline. A *snapshot* is a record of the current measurements, which the tool writes every N references. The sequence of snapshots of one run is its *timeline*, and the live footprint over the timeline is the run’s *footprint curve*.

Sample and sampling interval. A *sample* is a subset of the references (or of the addresses) of a run, chosen at random. A sample with *sampling interval* k , or sampling rate $1/k$, contains each element with probability $1/k$. We write “1 in k ” for such a sample.

Allocation, block and page. An *allocation* is a request for memory, made to the C library’s memory allocator, for example by calling `malloc`, or to the operating system, by calling `mmap`. A *block* is the memory range that one allocation returns. The operating system manages memory in *pages*, which are 4KB on our machine, and a block can start and end inside a page. A page is *resident* when the operating system has assigned physical memory to it. For memory that is not backed by a file, which the operating system calls anonymous memory, this happens at the first read or write of the page.

Native time and overhead. The *native time* of a run is its running time without any instrumentation. The *overhead* of a tool is the running time with the tool divided by the native time, written for example as $3\times$.

Ground truth. The *ground truth*, or truth, of an input is the footprint curve measured from a complete recording of every reference of a run on that input.

3 Background

3.1 Dynamic binary instrumentation with Pin

Pin [25, 22] runs a program in the same process as a *Pin tool*, which is a library that tells Pin what to observe. Pin does not run the program’s machine code directly. Instead, just before a piece of code runs for the first time, Pin translates it into a new version that also calls the tool, stores the new version in a memory area called the *code cache*, and runs the new version from there. The unit that Pin translates is a *trace*, which is a straight sequence of instructions that is entered only at its first instruction and may be left at several points. A trace consists of *basic blocks*, each of which is entered only at its first instruction and left only at its last.

A Pin tool has two kinds of functions. An *instrumentation routine* runs once, when Pin translates a piece of code, and decides which calls to insert into the translation. An *analysis routine* is a function that the inserted calls invoke every time the translated code runs. For example, an instrumentation routine can insert, before every instruction that reads memory, a call to an analysis routine that receives the address read. Calling an analysis routine on every memory reference is what makes complete recording slow.

Pin provides several ways to make analysis cheaper. It can copy a short, simple analysis routine directly into the translated code instead of calling it, which is called *inlining*. It provides a pair of calls, which we call a *test and action pair*, in which a short test routine is inlined and a longer action routine runs only when the test returns true. It provides *trace buffers*, for which the tool describes a record layout, Pin writes one record per instrumented instruction into a buffer that belongs to the running thread, and Pin calls the tool when a buffer is full. Finally, a tool can discard all translations in the code cache with the function `PIN_RemoveInstrumentation`, after which Pin translates code again the next time it runs, and it can make a thread continue from a given machine state with `PIN_ExecuteAt`.

3.2 MemPrint

The MemPrint Pin tool has two modes.

Splitter. The *splitter* records every memory reference. It writes each reference (the instruction’s address, the start address, the size, and whether it is a read or a write) into a trace buffer of 1024 pages per thread, and when a buffer is full it processes the references in order. It keeps the exact footprint in a hash table from start address to largest size. In addition, for each sampling interval k in a list (by default 100, 250, 500, 750, 1000, 2500, 5000, 7500, 10000, 25000, 50000, 75000 and 100000), it keeps 20 *bins*, each of which is a footprint of its own. A reference enters an interval’s bins with probability $20/k$, decided by a linear congruential random number generator (a generator that computes $x \leftarrow 1664525x + 1013904223$ modulo 2^{32}), and then goes to one of the 20 bins chosen uniformly at random. Each bin therefore holds an independent 1-in- k sample of the references. The splitter’s outputs are the training data of MemPrint.

Sampler. The *sampler* records only 1 in i references. It decides with the same random number generator, in an inlined test, whether a reference is sampled, so that a reference that is not sampled costs only the test. It places each sampled reference into K distinct bins out of s , where K is drawn from a Poisson distribution with mean $\lambda = s/r$, using Knuth’s method of multiplying uniform random numbers until the product falls below $e^{-\lambda}$. Drawing a Poisson number of copies of each element is a way of building bootstrap samples [13] from data that arrives one element at a time [19]. A bin then holds a sample with an effective interval of $i s / E[\min(K, s)]$, which is about $i r$ when λ is much smaller than s .

The α model. MemPrint defines $\alpha = F/F_{\text{bin}}$, where F is the true footprint and F_{bin} is the footprint of one bin, and it models $\log \alpha$ as a linear function of seven features of the bins. The features are $\log \sigma$, $\log k$ and $\log m$, where σ is the standard deviation of the footprint across the bins of an interval, k is the sampling interval and m is the bin footprint, together with the products $\log m \log \sigma$, $\log k \log \sigma$, $\log m \log k$ and $\log m \log \sigma \log k$. The model is fitted by least squares on the splitter’s bins of the small inputs of a program, using one of three subsets of the intervals: all of them (subset NZ), the intervals whose spread grows steadily with the input size (subset MT), or the best such interval together with one smaller and three larger intervals (subset L2O). The model is then applied to the bins of a sampler run on a large input.

Reproduction. We reorganized the original scripts into one Python package and reproduced all 16 data figures of the MemPrint manuscript pixel for pixel, together with its tables. The only difference is the average row of the manuscript’s error table, which was computed from an earlier version of the data.

4 Measuring the live footprint over time

We added three options to the Pin tool, which we call `memprint_trace`. With all three options turned off, the tool’s output files are byte for byte identical to those of the original tool, which we checked on PolyBench with fixed random seeds and with address space layout randomization turned off. Appendix A lists all options.

4.1 Removing freed memory

With `-track_frees 1`, the tool removes freed memory ranges from every footprint it keeps. To find the ranges, it inserts calls at the entry and at the exit of the C library’s allocation functions: `malloc`, `calloc`, `realloc`, `memalign`, `aligned_alloc`, `valloc`, `posix_memalign` and `free`. At the

entry of an allocation function it saves the requested size and, for `realloc`, the old address. At the exit it records the new block's address and size in a table. When a block is freed, the tool looks up its size and removes its range from the footprints. For `realloc`, the tool removes the old block if the block moved, or the part that was cut off if the block shrank in place. Several details were necessary to make this correct.

First, the GNU C library's `free` ends with a jump to another function rather than with a return instruction, so Pin never reports that it has returned. The tool therefore removes a block at the entry of `free`.

Second, allocation functions call each other, for example `calloc` may call `malloc`, so one call from the program can produce several entry events. The tool counts how deeply allocation calls are nested in each thread and records only the outermost call. It recognizes the outermost call by comparing the stack pointer at each entry with the stack pointer at the outermost entry, because a nested call runs with a lower stack pointer, whereas an entry at the same or a higher stack pointer starts a new outermost call.

Third, a function can be found more than once in the program's libraries. Calls to library functions go through small stubs in a section named `.plt`, which jump to the real function and never return, and some functions have several names at the same address, for example `memalign` and `aligned_alloc`. The tool skips stubs in `.plt` sections and instruments each function address only once.

Fourth, memory can also be returned to the operating system with the `munmap` system call, which some libraries call directly. The tool therefore watches every `munmap` system call, at its entry to learn the range and at its exit to learn whether it succeeded, and then removes the whole pages in the range.

To remove a range quickly, each footprint keeps, in addition to its table of addresses, an index from page number to the addresses recorded in that page. Removing a range then visits only the pages in the range, or only the recorded pages if there are fewer of them.

Ordering removals within a thread. In the splitter and the sampler, references wait in a thread's trace buffer until the buffer is full. A free must therefore not be applied to the footprints before the references that the thread made before the free. When a thread frees a block, the tool records the release together with the current fill position of the thread's buffer, and it applies the release when it has processed the buffer up to that position.

4.2 Snapshots and partial recordings

With `-snapshot N`, the tool appends the current state of every footprint it keeps to a timeline file every N references (Appendix B). Each row holds the time, the sampling interval, the bin number, the footprint in bytes, the number of distinct addresses, the number of references counted, the number of bytes freed so far, and five counts used in Section 6. The C library that Pin provides to tools took about 7 milliseconds to format one snapshot with its standard formatting functions, which slowed down runs with frequent snapshots, so the tool converts numbers to text itself.

With `-stop N`, the tool writes all its outputs after N references and detaches from the program with `PIN_Detach`, so that the program finishes without instrumentation. This option produces partial recordings.

4.3 Ordering events across threads

Because each thread has its own trace buffer, the tool processes the references of different threads in the order in which their buffers fill, not in the order in which the references happened. A free in one thread could therefore be applied before another thread’s earlier references to the same block, and those references then added the freed block back to the footprint. We found this error with a test in which 8 threads each allocate and touch a block, wait for each other with 12 MB in use, and then free their blocks. With buffers of 1024 pages, the complete recording reported a peak of only 9.8 to 10.8 MB. The tool now uses buffers of 16 pages when snapshots or free tracking are on, which limits how far the threads can drift apart, and the same test then gives 12.08 MB on every run. Without those options the tool keeps the original 1024-page buffers, so its default outputs do not change.

4.4 Tests

A test suite of small C and C++ programs with known footprints checks the tool. The programs allocate, touch and free blocks with `malloc` and `free`, grow and shrink blocks with `realloc` both in place and by moving them, use `calloc` and C++ `new`, map and partly unmap memory with `mmap` and `munmap`, allocate in one thread and free in another, use many small blocks, touch the same memory repeatedly, run 8 threads at once, and reuse freed heap memory. For each program, the suite checks the peak of the timeline, the number of bytes freed, the footprint at exit and the number of snapshots, and it also checks the `-stop` option.

5 Evaluation method

5.1 Workloads, inputs and machine

PolyBench. PolyBench/C 4.2.1 [34] is a collection of small numerical kernels. We use four of them: 2mm (two matrix multiplications), gemm (one general matrix multiplication), jacobi-2d (a two-dimensional stencil, which updates each point of a grid from its neighbors, repeated for many time steps) and atax (a matrix and vector product). As in MemPrint, we compile them without optimization. We added three sizes between the standard ones, so that the sizes MINI, MINI2, MINI3, SMALL, SMALL2, SMALL3 and MEDIUM are available for the studies of Sections 6 and 7, and we use the standard LARGE size, whose live peaks are 27 to 37 MB and whose runs make 28 to 49 billion references, for windowed sampling. PolyBench allocates its arrays at the start of a run and frees them at the end.

miniVite. miniVite [17] is a small application that finds communities in a graph with the Louvain method [5], which repeatedly moves each vertex to the neighboring community that most improves a quality score. It generates a random geometric graph, which connects points that lie close to each other in the unit square. It uses MPI for communication between processes and OpenMP [31] for threads within a process. We run one MPI process with 1, 4 or 16 OpenMP threads, on graphs of 1024 to 65536 vertices, and we set `OMP_WAIT_POLICY=passive`, so that idle threads sleep instead of repeatedly checking for work. miniVite allocates and frees memory throughout its run: at 65536 vertices it makes about 8 million calls to `malloc` and `free`.

Building miniVite. We first built miniVite against the OpenMPI [16] library of the operating system. That build touched about 175 MB at every graph size, mostly in buffers that the MPI

library sets up at the start, and its threads, which repeatedly checked for work while waiting, made 97% of its 1.76 billion references at 1024 vertices. We rebuilt it against OpenMPI 5.0.5 from the Spack package manager, the version used for the MemPrint manuscript. With that build and with sleeping idle threads, a complete recording at 1024 vertices gives a total footprint of 6.8 to 7.0 MB, which matches the 6.84 MB in the manuscript, from 44 million references.

Machine. All runs use Pin 3.30 on a machine with two Intel Xeon E5-2620 v4 processors (16 cores and 32 hardware threads in total) and 62 GB of memory, running Rocky Linux 8.10 with Linux 4.18 and GCC 8.5.

5.2 Ground truth and error metrics

We obtain the ground truth of an input from a splitter run with `-track_frees 1`, with the same number of threads as the run being evaluated.

We compare an estimated curve $\hat{F}(t)$ with the true curve $F(t)$ using three numbers, each expressed in percent. Because two runs of a multithreaded program do not execute exactly the same number of references, we compare times as fractions of each run, so that the estimate at 40% of its run is compared with the truth at 40% of its run.

- The *mean error* is the mean, over the snapshots of the estimated run, of $|\hat{F}(t) - F(t)|/F(t)$. It is also known as the mean absolute percentage error.
- The *error of the peak* is $(\max_t \hat{F}(t) - \max_t F(t))/\max_t F(t)$. It compares the largest estimate with the largest true value, wherever each occurs.
- The *error at the peak* is $(\hat{F}(t^*) - F(t^*))/F(t^*)$, where t^* is the time at which the true footprint peaks.

Training and testing. Estimators that need training are trained, as in MemPrint, on the smaller inputs of a program and tested on an input that was not used for training. We test on the largest input, which we call *extrapolation*, and on the middle input, which we call *interpolation*, each time training on all the other inputs.

6 Estimating from sampled references

Our first approach applies the idea of MemPrint at every snapshot: estimate the live footprint at time t from the sampled references made up to t . Table 1 summarizes the estimators we tried, and Figure 1 shows them on the MEDIUM input of 2mm.

6.1 The α model at every snapshot

We fitted the α model of Section 3.2 at every snapshot of the training inputs, using the splitter’s bins with frees tracked, and applied it to every snapshot of a sampler run. The estimate has 45% mean error for extrapolation and 31% for interpolation. A bin holds between 0.1% and 10% of the footprint and almost never contains the same address twice. A model that works from one bin therefore cannot tell memory that is used for the first time from memory that is used again. As a result, when the true footprint stops growing, the estimate keeps rising, because every new sampled reference looks like new memory.

6.2 Estimators of unseen species

The number of distinct addresses that a sample has never seen is an instance of a classic problem in ecology: estimating how many species exist from a sample of animals, some of whose species were never caught [18, 7]. These estimators use *frequency counts*: f_1 is the number of addresses sampled exactly once, f_2 the number sampled exactly twice, and so on. We changed the tool so that every footprint keeps, for each address, the number of times it was sampled, together with running totals of f_1 to f_4 . We also changed the splitter so that it keeps the union of the 20 bins of each interval, which is a 1-in- $k/20$ sample that contains enough repeated addresses.

The Chao1 estimator [8], in its form corrected for small samples, estimates the number of distinct addresses as $S + f_1(f_1 - 1)/(2(f_2 + 1))$, where S is the number of addresses seen. The iChao1 estimator [9] adds the term $\frac{f_3}{4f_4} \max(f_1 - \frac{f_2 f_3}{2f_4}, 0)$, which uses f_3 and f_4 to correct the case in which some addresses are used much more often than others. We convert an estimated number of addresses to bytes by multiplying it by the observed number of bytes per address.

Both estimators stop the estimate from rising on a plateau. For example, the error for jacobi-2d extrapolation falls from 42% to 52% down to 4% on the splitter’s unions and to 9% on the sampler. However, both are lower bounds when some memory is used once and other memory is used many times, so they underestimate kernels with many addresses written only once by 30% to 60% at moderate sampling rates. At the densest unions they overshoot by 50% to 245%, because f_2 is then much smaller than f_1 .

6.3 An estimator that uses the known sampling rate

Unlike an ecologist, we know the sampling rate p exactly, because it is the number of sampled references divided by the number of references executed, and we also know the number of references executed, T . We use both. We model the number of references made to one address as $1 + X$, where X follows a negative binomial distribution with mean m and shape a . Depending on its shape, this distribution can describe memory that is mostly used once as well as memory that is used many times. If each reference is sampled independently with probability p , the number of times an address appears in the sample has the probability generating function $(q + pz) B(z)$, where $q = 1 - p$, $B(z) = ((1 + c) - cz)^{-a}$ and $c = mp/a$. The probability of seeing an address exactly j times is therefore

$$P(j) = q b_j + p b_{j-1}, \quad b_j = (1 + c)^{-a} \binom{a + j - 1}{j} \theta^j, \quad \theta = \frac{c}{1 + c}. \quad (1)$$

Two quantities are known exactly: the total number of references, $U(1 + m) = T$, where U is the number of distinct addresses, and the number of addresses seen, $U(1 - P(0)) = S$. For a given shape a , these two equations determine U and m , which we solve numerically with Brent’s root-finding method. We choose the shape among 40 values between 0.02 and 200 by how well $UP(1), \dots, UP(4)$ and $U(1 - P(0) - \dots - P(4))$ match f_1 to f_4 and $S - f_1 - \dots - f_4$, measured by Pearson’s chi-square distance.

On dense samples (1 in 5 references) the known-rate estimate is within +1% to −12% of the truth, against +53% to +245% for iChao1, and at 1 in 12 it is within about 10% to 20%. At 1 in 50 and sparser, however, f_2 to f_4 are too small to identify the shape, and the error ranges from −90% to +215%. Fixing the shape from the training inputs does not help (40% to 150% error), because the fitted shapes are small (0.02 to 0.08), which indicates a mixture of memory that is used once and memory that is used many times, rather than one smooth distribution.

While evaluating this estimator, we also found that the union of a sampler’s bins does not have the sampling rate $1/i$. A sampled reference enters the union only if it is placed in at least one bin,

Table 1: Estimators that work from sampled references, on four PolyBench kernels, with the largest (extrapolation) and middle (interpolation) of the sizes MINI to MEDIUM held out. Values are the mean absolute error over the four kernels, in percent.

Estimator	Sample	Extrapolation			Interpolation		
		Mean	Of peak	At peak	Mean	Of peak	At peak
α model at every snapshot	1/25, 1/50	45	30	58	31	59	49
Chao1	1/25	28	9	33	57	39	60
iChao1	1/25	27	11	33	55	36	58
Known rate	1/3	12	26	17	7	23	10
Hybrid	1/3	6	11	12	5	9	8
Hybrid, with all four changes	1/25	24	21	38	9	12	24
Address sampling (Section 7)	1/25	0.9	0.8	0.8	3.4	3.4	3.4

which happens with probability $1 - e^{-\lambda}$, so the union’s rate is $(1 - e^{-\lambda})/i$. Our first results had used $1/i$, which overstated the rate by a factor of 1.6.

6.4 A hybrid of the known-rate estimate and a learned correction

The known-rate estimate errs in a consistent direction: it underestimates while addresses have not yet been used again. We therefore learn a correction on the training inputs. Specifically, we fit a ridge regression [20], which is least squares with a penalty on the size of the coefficients (here with weight 1), for $\log(F/\hat{F}_{\text{known}})$ from eight features: how far the known-rate estimate extrapolates beyond the observed footprint, $\log((f_2 + 1)/(f_1 + 1))$, the fraction of sampled references that were new addresses, f_1/S , the sampling rate, and the bins’ standard deviation, mean footprint and sampling interval. We fit the correction on the splitter’s unions at the sampling rate closest to that of the run being estimated.

With a dense sampler (1 in 3 references, which costs about 40% to 55% of a complete recording), the hybrid estimate has 6% mean error for extrapolation and 5% for interpolation. At 1 in 25 it has 24% and 9%.

We then tried four changes at 1 in 25. First, we trained the correction at exactly the sampler’s rate, by running the splitter with matching intervals. Second, we made the estimate non-decreasing between frees, using isotonic regression [2] (the closest non-decreasing curve in the least squares sense) on each segment between two frees, because the live footprint can only decrease when memory is freed. Third, we fitted one reuse shape per run instead of one per snapshot. Fourth, we added the fraction of new addresses among the samples of the last five snapshots, using a new counter of how often an address entered the sample. The extrapolation error stayed between 23% and 25% after each change. Replacing the features measured in bytes with ratios that do not depend on the input size did not reduce it either. The remaining error is a bias with input size. In 2mm and gemm, each array element is used about N times for an $N \times N$ matrix, so a larger input shows more repeats per address at the same sampling rate than any training input did, and a correction learned on smaller inputs does not apply.

Forecasting. We also tried to predict the rest of a footprint curve from its beginning. We stretched each training input’s curve in time and in size to match the observed beginning, and averaged the stretched curves with weights that decrease with how badly each one fitted. The predicted peak was within about 10% to 20% for interpolation but unreliable for extrapolation,

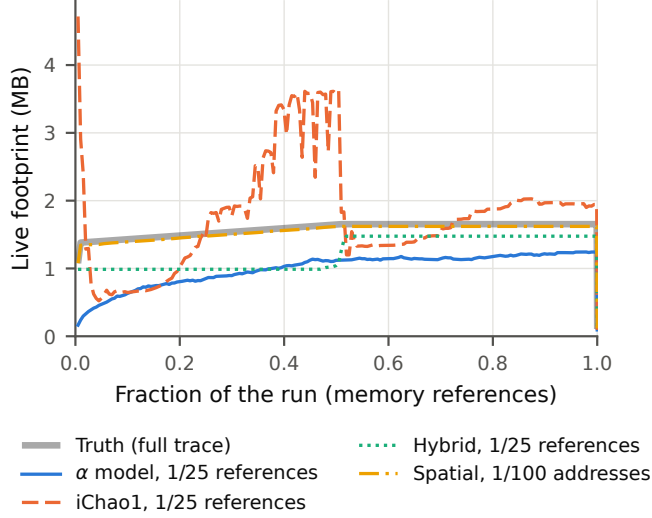


Figure 1: Live footprint of 2mm MEDIUM (held out from training) over the run, against estimates from 1-in-25 samples of references and from a 1-in-100 sample of addresses. The address sample follows the truth closely, whereas the estimates from sampled references overshoot, undershoot or lag.

and we did not pursue forecasting further.

7 Sampling addresses

All the estimators of Section 6 fail for the same reason. An address that is referenced r times appears in a sample of references with probability $1 - (1 - p)^r$, which depends on r , and r is unknown and grows with the input size. Sampling addresses instead of references removes that dependence, because an address is then in the sample with a probability that does not depend on how often it is used.

7.1 Method

In the tool’s *spatial* mode (`-mode spatial`), an address a is *selected* when $h(a \oplus \sigma) < 2^{64}/i$. Here $\oplus$ is the bitwise exclusive or, σ is a 64-bit number (a salt) derived from the run’s random seed, and h is the final mixing function of the splitmix64 random number generator [38], which scrambles the bits of its input so that nearby addresses, or addresses at regular distances, give unrelated results. Every reference to a selected address is recorded, and no reference to another address is.

Each address is selected with probability $1/i$, independently of how the program uses it. Let $F_S(t)$ be the footprint of the selected addresses at time t . Every address in the true footprint contributes its size to F_S with probability $1/i$, so the expected value of $F_S(t)$ is $F(t)/i$, and

$$\hat{F}(t) = i F_S(t) \quad (2)$$

is an *unbiased* estimate of $F(t)$, meaning that its average over many choices of the salt equals the truth. It is the Horvitz–Thompson estimator [21] for a sample in which every element has the same probability of being included [11]. No model and no training are needed. The number of selected addresses follows a binomial distribution, so for U live addresses of similar sizes, the standard

deviation of the estimate relative to the truth is about $\sqrt{(1 - 1/i) i/U} \approx 1/\sqrt{U/i}$. For example, 200,000 addresses sampled at 1 in 100 give about 2%.

To report an error bar, the tool also splits the selected addresses into 20 *buckets* by the value of $h(a \oplus \sigma) \bmod 20$. Each bucket is an independent 1-in-20*i* sample, so $20 i F_{\text{bucket}}$ is an independent estimate of F , and the standard deviation of the 20 bucket estimates divided by $\sqrt{20}$ is a standard error of $\hat{F}$.

Implementation. The selection test is an inlined test routine that receives the reference’s address, adds one to the running thread’s reference counter, and returns whether the address is selected. A selected reference calls an action routine that records it immediately in the footprint and in its bucket, while holding a lock shared by all threads. Because selected references are rare, recording them immediately costs little, and it keeps the order of references and frees across threads exact, because the tool also applies frees immediately under the same lock. The time of a snapshot is the sum of the threads’ reference counters.

We first recorded selected references through trace buffers, as the splitter does, which brought back the ordering problem of Section 4.3. We then held each free back until every thread had processed all the references it had made before the free, using vector clocks, which record for each thread how many events of every other thread it has seen. This approach was correct, but miniVite frees memory constantly while its MPI helper threads rarely fill a buffer, so a run at 8192 vertices took 7474s. Recording selected references immediately took 28 to 40s for the same run, and the 8-thread test of Section 4.3 then gives 12.03 to 12.27 MB, against a truth of 12.09 MB.

7.2 Results

PolyBench. With the largest and middle of the sizes MINI to MEDIUM held out, as in Section 6, address sampling has 0.9% and 3.4% mean error at 1 in 25 addresses, 2.3% and 4.2% at 1 in 100, 2.3% and 9.9% at 1 in 250, and 6.7% and 17.5% at 1 in 1000 (Figure 2). The error follows the binomial prediction. The largest sizes (130,000 to 200,000 addresses) stay within 1% to 7% down to 1 in 1000, whereas the middle sizes (about 28,000 addresses) need 1 in 100 or denser, because at 1 in 1000 only about 28 of their addresses are selected. The true footprint lies within two standard errors of the estimate in 100% of snapshots at 1 in 25 and 1 in 100, and in 88% and 77% of snapshots at 1 in 250 and 1 in 1000, against 95% expected, because a standard error computed from 20 buckets is itself uncertain.

miniVite and threads. With frees tracked, the live peak of miniVite is 4.9, 6.2 and 7.9 MB at 1024, 4096 and 8192 vertices, the same at every thread count. Address sampling has 0.2% to 6.1% mean error at every rate from 1 in 25 to 1 in 1000 with 1, 4 and 16 threads, the error of the peak is at most 2.8%, and the truth lies within two standard errors in 94% to 100% of snapshots. At 8192 vertices a run takes 26 to 41 s, against 168 to 198 s for a complete recording and about 0.25 s natively.

Cost on large inputs. On the LARGE PolyBench inputs, address sampling at 1 in 100 takes 211 s for 2mm, 215 s for gemm and 236 s for jacobi-2d, against 7.5, 6.0 and 11.3 s natively, which is an overhead of 21× to 36×. Pin without a tool takes 8.1, 6.1 and 11.6 s, so almost all of the cost is the selection test on every reference.

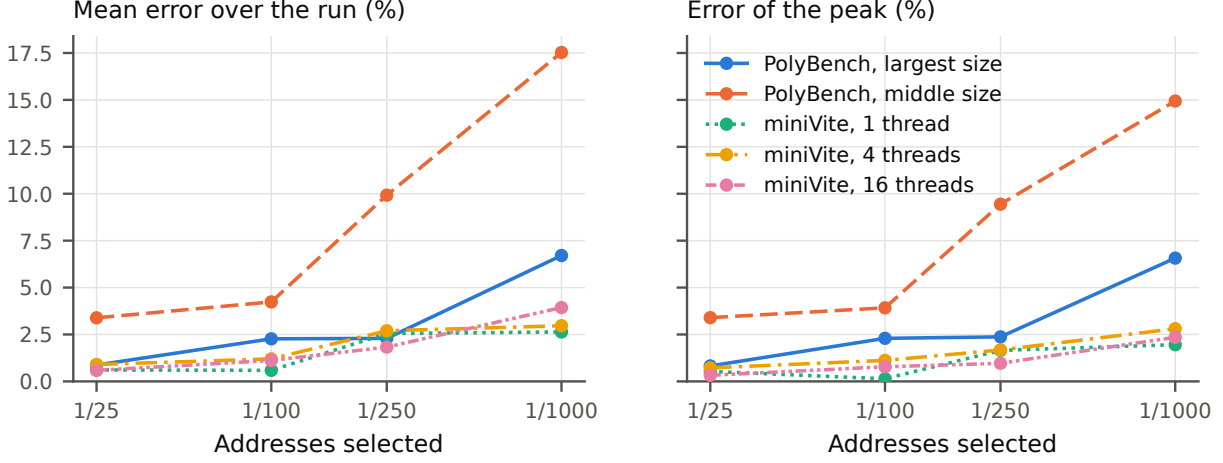


Figure 2: Address sampling error against the fraction of addresses selected. PolyBench values are the mean over 2mm, gemm, jacobi-2d and atax for the held-out largest and middle of the sizes MINI to MEDIUM. miniVite values are the mean over 1024, 4096 and 8192 vertices.

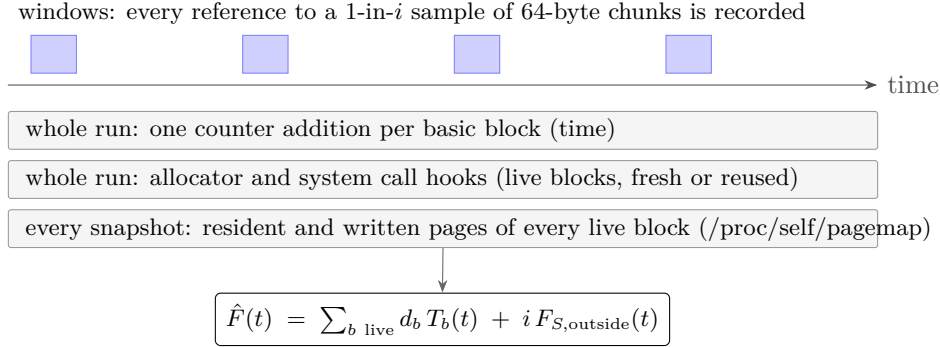


Figure 3: Windowed sampling. References are checked only inside windows (shaded). Time, live blocks and page information are tracked for the whole run. T_b is the touched bytes of block b (resident bytes for a fresh block, bytes on written pages for a reused block), d_b is its density, and $F_{S,\text{outside}}$ is the selected footprint outside all blocks.

8 Windowed sampling

Address sampling still runs the selection test on every reference, which costs $21\times$ to $36\times$ native time on the LARGE PolyBench inputs, whereas Pin without a tool costs almost nothing. For long programs, therefore, the test itself is the obstacle, not the sampling rate. *Windowed sampling* (options `-window W` and `-period P`) runs the selection test only during *windows* of W references at the start of every P references, so that a fraction W/P of the run is *watched*. Between windows it runs only things that cost little and that still observe the whole run. Figure 3 gives an overview, and the rest of this section describes each part.

8.1 Counting time between windows

Between windows the tool must still count references, because time is measured in references, and windows start and end at given times. Counting each reference separately would cost almost as much as the selection test. Instead, when Pin translates a basic block, the tool counts the block’s

memory references, which is the number of memory operands that the block’s instructions read plus the number that they write, and it inserts one inlined call at the start of the block that adds that number to the running thread’s counter. The same call compares the counter with a threshold and, only when the threshold is reached, calls a longer routine that advances time for all threads (Section 8.2).

On miniVite with one thread, this count matches the splitter’s count of every reference to within 0.0001% over 1.78 billion references. The two counts differ in two cases. An instruction with a REP prefix, which repeats a string operation many times, counts once rather than once per repetition, and a basic block that is left before its end, for example because of a signal, still counts all its references.

Each thread’s counter sits in its own 64-byte memory area, which is the unit in which processors cache memory, so that threads do not slow each other down by writing into the same cached unit.

8.2 Switching between watching and not watching

When a thread’s counter passes its threshold, the thread takes the shared lock, adds up all threads’ counters to get the current time, writes any snapshot that is due, and opens or closes a window if the time has passed a window’s start or end. It then sets its next threshold s references ahead, where

$$s = \max\left(10^4, \frac{1}{8} \min(W, P - W, N)\right), \quad (3)$$

so that every thread checks the time several times within the shortest of a window, a gap between windows and a snapshot interval, but not so often that the lock becomes expensive. The first version used 10^3 instead of 10^4 , and a run with 1000-reference windows then spent most of its time taking the lock.

When a window opens or closes, the instrumentation must change, because inside a window every reference needs the selection test, and outside a window it does not. The tool calls `PIN_RemoveInstrumentation`, after releasing the lock, which discards all translations, so that Pin translates code again, with the new instrumentation, the next time it runs. However, discarding translations does not change code that is already running. A loop keeps jumping back to the start of its own translation, so it keeps running the old instrumentation until it leaves the loop. In 2mm, a window that had closed kept the selection test running for the rest of a loop nest, so a run that should have watched almost nothing took 70s instead of 19s, and windows did not start or stop on time.

The tool therefore counts window changes in a shared counter, and each thread remembers the last change it has acted on. When a thread checks the time and finds a change it has not acted on, it restarts at its current instruction with `PIN_ExecuteAt`, using the machine state that Pin passes to the routine, so that Pin translates the code it is running again. Threads check the time every s references also while running old code, because the counting call is present in both versions of the code. A reference that old code records after a window has closed is discarded.

8.3 Tracking live blocks

In windowed mode the allocator hooks of Section 4.1 are always on, and the tool keeps every live block in an ordered map from start address to size, so that it can find the block that contains a given address. It also records memory that the program maps directly. When an `mmap` system call that maps anonymous memory succeeds outside an allocation call, the tool adds the mapping as a block, and when `munmap` unmaps part of such a block, the tool removes that part and keeps the

rest. When `realloc` grows or shrinks a block in place, the tool keeps the block and changes its size.

The tool classifies each block as *fresh* or *reused*. A block is fresh when its pages were mapped for it, which is the case for an anonymous mapping made outside the allocator, and for an allocation during which the allocator called `mmap` or `mremap`. The tool knows the latter because it watches system calls and knows which thread is inside an allocation call. By default, the GNU C allocator serves requests of 128 KB or more by mapping new memory [15], so large arrays are usually fresh. All other blocks are reused, because the allocator takes them from memory it already owns, which may have been used by blocks that were freed earlier.

8.4 Page residency

A fresh block's pages are untouched when the block is created, and a page becomes resident at its first read or write. The number of a fresh block's bytes that lie on resident pages is therefore the number of bytes the block has touched, rounded to whole pages, whether or not a window is open.

The tool reads residency from `/proc/self/pagemap` [39], a file through which Linux reports one 64-bit entry per page of the process. In an entry, bit 63 is set if the page is resident, bit 62 if it has been moved to swap space, and bit 55 if it has been written since the last reset of that bit (Section 8.5). At every snapshot the tool visits the live blocks in address order, reads the entries of their pages in chunks of 512 pages so that neighboring blocks share reads, and for each block adds up the bytes of the block that lie on resident or swapped pages. At a block's first and last page it counts only the bytes inside the block. Pin's C library does not provide the `pread` function, so the tool uses `lseek` followed by `read`.

Our machine enables transparent huge pages [41], which let the operating system back a 2 MB region with one large page at its first touch. A single touch can then make a whole 2 MB region resident. In a test that wrote 16 MB of a 64 MB block, `pagemap` reported 18.4 MB resident with huge pages on. The tool therefore turns off huge pages for the traced process with `prctl(PR_SET_THP_DISABLE)`, through the raw system call because Pin's C library does not provide `prctl`. With huge pages off, the same test reports exactly 16 MB, and reading another 8 MB adds exactly 8 MB, because a read of untouched memory also makes its page resident.

8.5 Written pages of reused blocks

The residency of a reused block can include pages that earlier blocks at the same addresses touched, because memory that a program frees stays resident and the allocator hands it to later blocks. In a test that keeps 64 heap blocks live, replaces the oldest one at every step, and writes only the first quarter of each new block (Section 9.4), residency overestimates the live footprint by 248%.

The tool therefore measures a reused block by the pages written since the block was allocated, using the kernel's soft-dirty bits [40]. Writing the character 4 to the file `/proc/self/clear_refs` clears the soft-dirty bit of every page of the process, and after that, bit 55 of a page's `pagemap` entry is set if the page has been written. Memory that the allocator returns is not initialized, so a program writes heap memory before it reads it, and the pages a reused block has written since its allocation are the pages it has touched. Each reused block keeps one mark per page, and at every snapshot the tool sets the mark of each of its pages whose soft-dirty bit is set. The touched bytes of a reused block are its bytes on marked pages.

To clear the soft-dirty bits, the kernel removes write permission from every page, so the next write to each page causes a fault, in which the kernel sets the bit and restores the permission. Each clear therefore costs one fault for each page that is written afterwards, and the tool clears the bits

only at three kinds of events. Before every clear, it first sets the marks of every live reused block from the current bits, so that no write is lost.

1. At the entry of an allocation call of 16 KB or more. Bits that earlier blocks set on the new block’s pages are then cleared before the block exists. The clear happens at the entry rather than at the exit because `calloc` sets the new memory to zero and `realloc` copies the old block’s contents inside the call, and those writes belong to the new block. Smaller blocks share pages with other blocks, so clearing for them would add cost without making their measurement exact.
2. Around the `brk`, `mmap` and `mremap` system calls. When the kernel creates or grows a memory region, it marks the whole region as soft-dirty, so that all its pages read as written until the next clear, whether or not they were written. The allocator grows the heap with `brk`, and before we handled this case, the written bytes of the churn test jumped between 1.3 MB and 3.7 MB from one snapshot to the next. The tool now sets the marks at the system call’s entry and clears the bits at its exit.
3. Every `-dirty_every` references, by default every ten snapshots.

A block whose pages are all marked cannot change any more, so the tool skips it when it sets marks before a clear. On miniVite with one thread, which clears the bits 8509 times, this reduced the running time from 138 s to 86 s.

8.6 Chunk sampling and density

Residency and written pages count bytes, but the footprint counts the largest access at every start address, so overlapping accesses count more than the bytes they cover (Section 2). For example, a copy loop that reads 16 bytes at a time, starting every 8 bytes, adds 16 bytes to the footprint for every 8 bytes it covers. We call the footprint divided by the bytes covered the *density*. In miniVite’s reused blocks the density is about 1.45 for most of the run and about 1.1 at the end, so an estimate from touched bytes alone is about 20% too low at the peak.

The windows measure the density. To do that, they must see, for some part of memory, both the footprint and the bytes covered. A sample of single addresses cannot provide the bytes covered, because the neighbors of a selected address, whose accesses may overlap it, are usually not selected. Inside windows the tool therefore selects 64-byte *chunks* instead of single addresses, with the test $h(\lfloor a/64 \rfloor \oplus \sigma) < 2^{64}/i$, and records every reference whose start address lies in a selected chunk. Every address is still selected with probability $1/i$, so iF_S remains an unbiased estimate of the footprint of the references watched.

At every snapshot during a window, and at the end of every window, the tool lists the selected start addresses (with their largest sizes) that lie inside live blocks, sorts them by address, and walks through them to compute, for each block, the selected footprint $F_{S,b}$, which is the sum of the sizes, and the bytes covered $C_{S,b}$, which is the length of the union of the ranges $[a, a + \text{size})$. The density of block b is

$$d_b = \begin{cases} F_{S,b}/C_{S,b} & \text{if } iC_{S,b} \geq 64 \text{ KB,} \\ F_{S,\text{reused}}/C_{S,\text{reused}} & \text{otherwise, for a reused block, if } iC_{S,\text{reused}} \geq 64 \text{ KB,} \\ 1 & \text{otherwise,} \end{cases} \quad (4)$$

where $F_{S,\text{reused}}$ and $C_{S,\text{reused}}$ are the totals over all reused blocks. A fresh block falls back to 1 rather than to the reused blocks’ density, because the large arrays that fresh blocks usually hold

are written in whole words. The density does not depend on how much of the run the windows covered, because the footprint and the bytes covered are measured on the same chunks.

8.7 The estimate

At every snapshot the tool writes one row to a separate output file (Appendix B), which includes the estimate

$$\hat{F}(t) = \sum_{b \text{ live}} d_b(t) T_b(t) + i F_{S,\text{outside}}(t), \quad (5)$$

where T_b is the touched bytes of block b (resident bytes for a fresh block, bytes on written pages for a reused block), d_b is its density, and $F_{S,\text{outside}}$ is the selected footprint outside every block, which covers the stack, global variables and memory mapped from files, and which only windows see. The row also contains every part of the estimate, so that other combinations can be evaluated without running the program again.

8.8 Approaches that did not work

We arrived at (5) after several approaches that failed, which we list in the order in which we tried them.

1. **The windows’ sample alone.** Multiplying the selected footprint by i misses every address that is touched only between windows. On 2mm LARGE at 1% watched it has 45% mean error.
2. **A model per allocation site.** We estimated a block from the selected bytes that windows had seen in it, and a block that no window had yet seen from the fraction touched in earlier blocks allocated at the same place in the program. This estimate also missed every touch made between windows, and it had 29% to 47% mean error on miniVite at 1% watched and 8% to 25% at 5%.
3. **Residency for all blocks.** Residency is exact for fresh blocks but overestimates reused blocks that sit on memory used before, by 248% on the churn test.
4. **A density from sampled pages.** Dividing the selected footprint by the resident bytes of the pages that contained a selected address was noisy, and one run ended with a density of 2.7, because a page with few touched addresses enters the calculation only when one of them is selected.
5. **One density for all reused blocks and none for fresh blocks.** miniVite with 32768 vertices reaches its peak while a fresh block is written with overlapping accesses (Section 9.3), so this version left that peak 12% to 17% too low.

9 Evaluation of windowed sampling

We compare windowed runs with a complete recording of the same input and thread count. All runs select 1 in 100 chunks and use about 20 windows per run (the period is about one twentieth of the run), and we did not tune any setting for any workload. The inputs are 2mm, gemm and jacobi-2d LARGE, miniVite with 32768 vertices (6.6 billion references, live peak 20 MB) and with 65536 vertices (25 billion references, live peak 36 MB), each with 1 and 4 threads, and a heap-churn test (Section 9.4). We compare (5) with three simpler estimates: *residency alone*, which is the

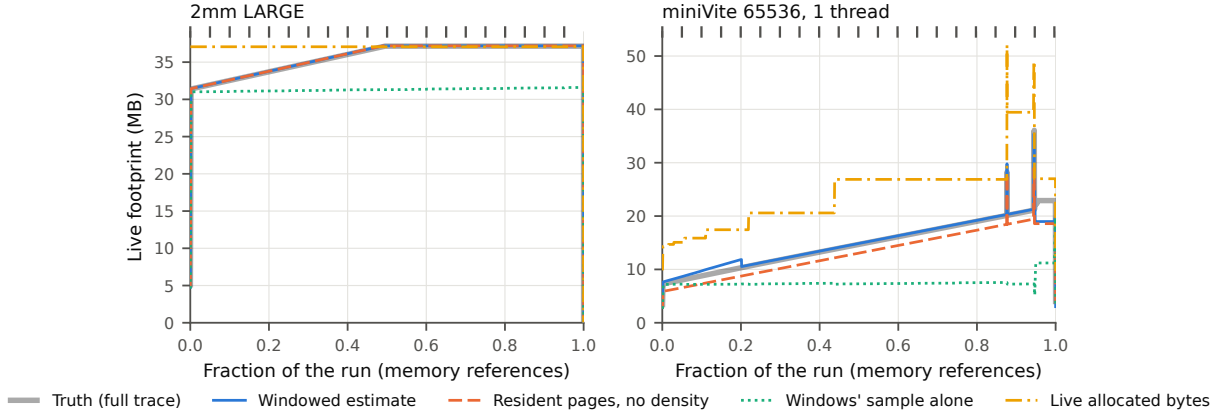


Figure 4: Live footprint over the run with 5% of the run watched (20 windows, marked at the top) and 1 in 100 chunks selected. On 2mm LARGE the windowed estimate and residency alone coincide with the truth.

resident bytes of all blocks plus the windows’ footprint outside blocks, without written pages or density; the windows’ selected footprint alone ($i F_S$); and the bytes of all live blocks, which we call the *allocated bytes*.

9.1 PolyBench

Every PolyBench array is allocated by mapping new memory, so all blocks are fresh and residency alone gives the exact curve. The windowed estimate is within 0.0% to 0.2% mean error and within 0.2% at the peak at every watched fraction, including none (Figure 4, left). The windows’ sample alone has 0.1% to 12.2% mean error at 5% watched and 10.5% to 45.3% at 1%, because it misses memory that is written once and then not used while a window is open. The allocated bytes have 0.4% to 4.5%.

9.2 miniVite

miniVite’s footprint grows in reused heap blocks for the whole run, and it allocates far more than it touches, so the allocated bytes overestimate it, with 67% to 107% mean error (Figure 4, right). Table 2 and Figure 5 show the windowed estimate. With 65536 vertices and 5% watched, it has 3.8% and 3.1% mean error with 1 and 4 threads, and -1.9% and -3.0% error of the peak, against about 12% mean error and -19% error of the peak for residency alone. At 1% watched, the density measured by the middle of the run is only 1.05 to 1.12, against about 1.45 at 5% and 20%, so the peak is about 15% too low. With nothing watched, the density stays 1 and memory outside blocks is not seen at all, so the peak is about 25% too low. Selecting 1 in 25 chunks instead of 1 in 100 changes the results by less than 3 percentage points.

9.3 The peak of miniVite with 32768 vertices

With an earlier version of the estimate, which applied one density to all reused blocks and none to fresh blocks, the peak of miniVite with 32768 vertices and 4 threads was 12% to 17% too low at 5% to 20% watched, whereas the same version was within 4% on 65536 vertices. We tested three possible causes in turn. First, we ran the same input with one thread. At the moment of the true

Table 2: Windowed estimate on miniVite, mean error / error of the peak in percent, 1 in 100 chunks, about 20 windows.

Input, threads	Nothing watched	1%	5%	20%
65536 vertices, 1 thread	25.9 / -24.6	9.1 / -14.5	3.8 / -1.9	4.5 / -0.4
65536 vertices, 4 threads	26.4 / -24.8	8.4 / -14.4	3.1 / -3.0	1.7 / -2.8
32768 vertices, 1 thread	32.1 / -27.9	14.4 / -14.1	8.3 / -9.8	4.3 / -4.3
32768 vertices, 4 threads	32.3 / -28.0	17.1 / -15.2	5.7 / -9.0	2.1 / -4.8

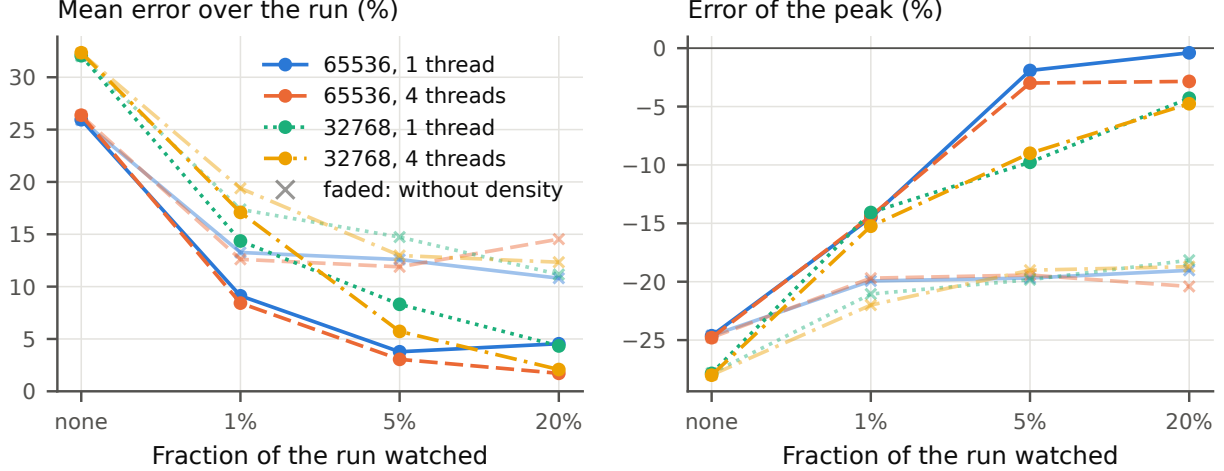


Figure 5: miniVite: windowed estimate error against the fraction of the run watched, with 1 in 100 chunks. Faded lines are residency alone, without written pages or density.

peak, the estimate was 17.6 MB with one thread and 17.5 MB with four, against a truth of 19.9 MB, so threads were not the cause. Second, we repeated the 4-thread run three times, and the errors agreed within 1.4 percentage points, so random variation was not the cause. Third, we used 40 and 80 shorter windows instead of 20, which made the error slightly worse, and we watched the whole run with 1 in 16 chunks, which still missed the peak by 12%, so the windows were not the cause.

The true peak is a short spike. At 90.5% of the run, 4.3 MB of new heap blocks appear and the true footprint rises from 13.3 to 19.9 MB, and about 0.3% of the run later it falls back. During the spike a fresh block is written with overlapping accesses. In the run that watched everything, that block’s selected footprint grew to 8.9 MB, against 6.75 MB of resident bytes, which is a density of about 1.3, and the earlier version counted it at 1. With a density per block (Section 8.6), the error of the peak is -9.0% and -4.8% at 5% and 20% watched with four threads, down from -17.2% and -11.9%, and -9.8% and -4.3% with one thread. At 5% watched, no window may overlap the spike, so the density of the block written during the spike is not always measured.

9.4 Heap churn

The heap-churn test keeps 64 live heap blocks of 16 to 112KB, which is below the allocator’s threshold for mapping new memory, so all blocks come from memory that the allocator reuses. At every step it frees the oldest block, allocates a new one, writes the first quarter (or half) of the new block, and reads the written part of one other live block. Its live footprint peaks at 1.3 MB when a quarter of each block is written and at 2.5 MB when half is written.

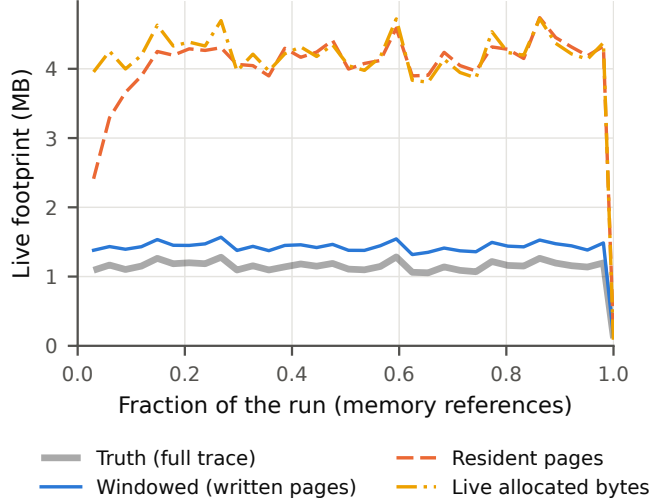


Figure 6: Heap churn with a quarter of each block written, 5% of the run watched. Resident pages include memory that freed blocks touched, whereas the pages written since each block’s allocation follow the truth to within whole pages.

Residency alone overestimates the footprint by 248% (mean error) when a quarter of each block is written and by 91% when half is written, because the pages of freed blocks stay resident and are handed to new blocks (Figure 6). Measuring reused blocks by their written pages reduces the error to 24% and 13%. The error is about the same at every watched fraction (22% to 26% when a quarter is written), because written pages, not windows, carry the estimate. The remaining error comes from whole pages. Each block’s written part ends inside a 4 KB page, which adds up to 4 KB per block, or about 0.25 MB for 64 blocks.

9.5 Cost

Table 3 and Figure 7 report running times measured one at a time on an otherwise idle machine, one run per configuration. On PolyBench LARGE, windowed sampling at 5% watched costs $2.8\times$ to $3.2\times$ native time, against $21\times$ to $36\times$ for address sampling of every reference, and with nothing watched it costs $1.9\times$ to $2.1\times$, which is mostly the cost of the per-block counter. Each further percent watched adds roughly one percent of the cost of address sampling. On miniVite with 65536 vertices it costs $13.0\times$ with one thread and $15.3\times$ with four at 5% watched, against $25\times$ and $27\times$.

The cost on miniVite is higher for three reasons. First, Pin itself costs $1.8\times$, because Pin translates the MPI library’s start-up code once and the run is short. Second, miniVite makes 8 million `malloc` and `free` calls at 65536 vertices, each of which runs the allocator hooks and takes the lock, and on miniVite with 16384 vertices, turning the allocator hooks off saved 7s of 22s. Third, soft-dirty tracking costs about 14% at 5% watched (84.3s against 73.7s without it, with one thread), and 4% to 14% on PolyBench. One run of jacobi-2d at 5% watched took 173s and a repeat took 31.2s, and we report the repeat. The complete recordings that provided the truth took 1237 to 1717s, but they ran at the same time as other runs, so they are not directly comparable.

Table 3: Running time in seconds, one run each, 1 in 100 addresses or chunks, about 20 windows.

Workload	Native	Pin, no tool	Address sampling	Windowed, fraction watched			
				none	1%	5%	20%
2mm LARGE	7.5	8.1	211	15.8	16.9	23.1	43.6
gemm LARGE	6.0	6.1	215	11.7	13.1	19.0	37.0
jacobi-2d LARGE	11.3	11.6	236	21.5	23.3	31.2	59.4
miniVite 65536, 1 thread	6.5	11.5	163	59.6	79.4	84.3	104.1
miniVite 65536, 4 threads	6.1	11.0	161	72.3	85.8	93.0	111.2

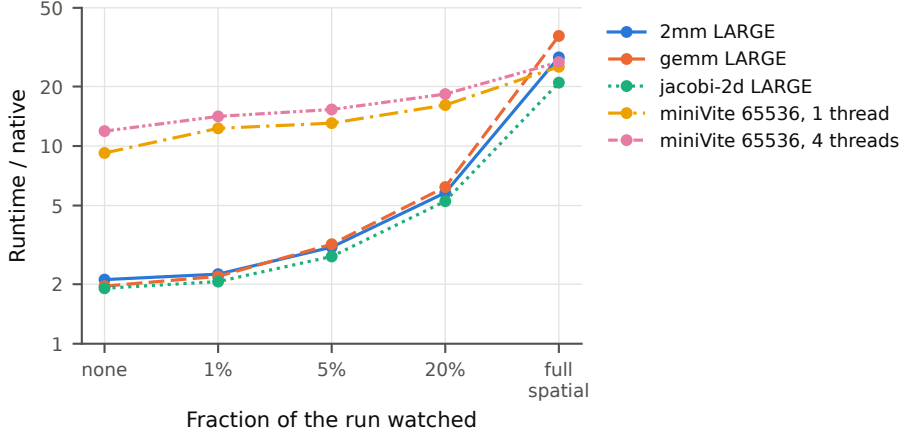


Figure 7: Running time relative to native time against the fraction of the run watched, with address sampling of every reference for comparison (logarithmic scale).

10 Discussion and limitations

Why each part is needed. Each part of (5) corrects one failure that we observed. Residency records first touches whether or not a window is open, which the windows’ sample alone cannot do. Written pages remove the pages that a reused block inherits from freed blocks, which residency alone cannot do. The density corrects the difference between the footprint, which counts overlapping accesses more than once, and the bytes touched. Selecting chunks rather than single addresses lets the windows measure the density independently of how much of the run they covered.

Limitations of written pages. A limitation of written pages is that they miss memory that a reused block only reads, which is rare for heap memory because the allocator does not initialize it, and that they are exact only to the page, which adds up to 4 KB per block and dominates for programs with many small blocks that are only partly touched (22% on the churn test). Blocks smaller than 16 KB do not clear the soft-dirty bits when they are allocated, so they can inherit marks from writes made since the last clear. With several threads, writes that other threads make between the entry and the exit of a `brk` or `mmap` call are cleared without being recorded.

Limitations of the density. A limitation of the density is that a block, or the set of reused blocks, needs a few windows’ worth of chunks before its density leaves the default value. As a result, the estimate is too low at 1% watched, and the peak of miniVite with 32768 vertices, which

lasts only 0.3% of the run, is 9% too low at 5% watched. A fresh block that no window has seen counts with density 1.

Other limitations. Memory outside blocks, such as the stack and global variables, is seen only in windows. The tool does not track `mremap` calls made outside `realloc`, or the growth and shrinking of the stack. Address and chunk selection depend on absolute addresses, which move when the tool library changes size, because the operating system places the program’s mappings around the tool’s, so spatial and windowed outputs are not byte for byte identical across builds of the tool, whereas splitter and sampler outputs are. Soft-dirty tracking requires a Linux kernel built with soft-dirty support, and the tool falls back to residency for reused blocks if `/proc/self/clear_refs` cannot be written.

Generality of the evaluation. We evaluated two applications and one synthetic test, and PolyBench uses only fresh memory. It remains unclear how the windowed estimate behaves on real programs that reuse their heap heavily, on long programs whose footprint changes in phases shorter than a window period, and with many MPI processes.

11 Related work

Measuring memory use. Valgrind [29] is a binary instrumentation framework, and its heap profiler Massif [36] records every allocation and reports the amount of heap memory reserved over time, at a high cost and without measuring which reserved bytes are used. Pin [25] and DynamoRIO [6] are binary instrumentation frameworks on which tools that record every memory access can be built, as MemPrint [28] does. The working set model [12] defines the memory that a program needs as the pages it used during a recent interval of time, which is a page-level, time-limited counterpart of the live footprint.

Hardware sampling of memory accesses. Intel’s Processor Event-Based Sampling (PEBS) and ARM’s Statistical Profiling Extension (SPE) let the processor record the address of a sampled memory access at low cost [30, 27]. MemGaze [24] analyzes traces of sampled loads, and HeMem [35] uses PEBS to decide which pages to place in fast memory. These approaches sample accesses, as the MemPrint sampler does, so an estimate of the footprint from them faces the bias of Section 6: frequently used memory is far more likely to be seen.

Page-level tracking by the operating system. Thermostat [1] samples pages and observes accesses to them by changing their page table entries, and DAMON [32] monitors how often memory regions are accessed in the Linux kernel by checking the accessed bits of sampled pages. Linux reports residency through `/proc/pid/pagemap` [39] and written pages through soft-dirty bits [40], which tools that save and restore processes use to find the pages changed since a previous save. We use residency and soft-dirty bits to measure first touches between windows, to the page, and we use sampled references to correct for what pages cannot show.

Sampling locations in caches and simulation. Simulating only a subset of cache sets, or only the addresses chosen by a hash, is an established way to make cache simulation cheaper [23]. SHARDS [42] keeps the references whose address hashes below a threshold to build miss ratio curves for storage caches from a small fixed fraction of the addresses, which is the same selection as our address sampling. Counter stacks [43] use probabilistic counters to estimate distinct elements and

miss ratio curves for storage traces. Reuse distance analysis [26, 4], StatCache [3] and StatStack [14] estimate cache behavior from sampled reuse, rather than the footprint over time.

Sampling over time. PinPoints [33] and SimPoint [37] choose representative regions of a run, SMARTS [44] simulates many short, regularly spaced samples of a run in detail, and instrumentation sampling for datacenter applications [10] turns instrumentation on and off over time, as our windows do. These methods estimate quantities that each region shows by itself, such as instructions per cycle. A footprint depends on the first touch of every address, which can happen outside every sampled region, which is why windowed sampling needs residency and written pages between windows.

Estimating unseen elements. Good [18] introduced the estimation of the probability of unseen species from frequency counts, and Chao1 [8] and iChao1 [9] estimate the number of unseen species [7]. Our known-rate estimator adapts these ideas to a sample whose rate is known exactly. Address sampling uses the Horvitz–Thompson estimator [21, 11], and the MemPrint sampler uses Poisson bootstrap weights [13, 19].

12 Next steps

We propose the following next steps, in order of priority.

1. **Open windows when large blocks are allocated.** Open a window when a large block is allocated, or when the rate at which pages become resident or written changes, instead of only at fixed times. The peak of miniVite with 32768 vertices lasts 0.3% of the run, and a window opened at the allocation that starts it would measure the density of the block written during it.
2. **Evaluate a real program that reuses its heap heavily.** The churn test is synthetic. A graph or sparse matrix code that allocates buffers in every iteration, or a language interpreter, would show whether written pages are enough in practice, and how the cost of clearing soft-dirty bits grows with the rate of allocation.
3. **Reduce the error from whole pages for small blocks.** For blocks smaller than a few pages, combine the written pages with the chunk sample, for example by scaling a block’s written bytes by the fraction of its selected chunks that were touched.
4. **Reduce the fixed cost of the allocator hooks and of soft-dirty clears.** On miniVite the allocator hooks account for about a third of the windowed running time, and soft-dirty tracking for about an eighth. Instrumenting the allocator in Pin’s probe mode, which patches the function’s entry instead of translating code, or recording allocation events in a per-thread log that is processed at snapshots instead of under a lock, would lower both.
5. **Evaluate at scale.** Run windowed sampling on a long application with several MPI processes, where a complete recording is not feasible, and compare the estimated peak with the maximum resident memory reported by the operating system, which is an upper bound to the page.
6. **Combine with hardware sampling.** PEBS or SPE could replace the per-block counter and the chunk sample at even lower cost, with the known-rate correction of Section 6.3 for their bias toward frequently used memory.

7. **Revisit forecasting.** Windowed sampling makes the beginnings of long runs cheap to record, which may make it practical to predict the rest of a run from the phases seen so far.

13 Conclusion

Estimating the memory a program uses over time from a random sample of its memory references fails at sparse sampling rates, because such a sample cannot separate memory used for the first time from memory used again. Sampling memory locations instead removes that difficulty and needs no training, but it still checks every reference. Checking references only during short windows, while tracking allocations, resident pages and, for reused memory, written pages for the whole run, keeps the estimate within 0.2% on PolyBench LARGE and within 3.1% to 3.8% on miniVite with 65536 vertices at 5% of the run watched, at $2.8\times$ to $3.2\times$ native time on PolyBench and $13\times$ to $15\times$ on miniVite. Measuring reused memory by the pages written since its allocation keeps the estimate within whole pages when freed memory is reused.

References

- [1] Neha Agarwal and Thomas F. Wenisch. Thermostat: Application-transparent page management for two-tiered main memory. In *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 631–644, 2017.
- [2] R. E. Barlow, D. J. Bartholomew, J. M. Bremner, and H. D. Brunk. *Statistical Inference under Order Restrictions: The Theory and Application of Isotonic Regression*. John Wiley & Sons, 1972.
- [3] Erik Berg and Erik Hagersten. Fast data-locality profiling of native execution. In *Proceedings of the 2005 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, SIGMETRICS '05, page 169–180, New York, NY, USA, 2005. Association for Computing Machinery.
- [4] Kristof Beyls and Erik H. D'Hollander. Reuse distance-based cache hint selection. In *Proceedings of the 8th International Euro-Par Conference on Parallel Processing*, Euro-Par '02, page 265–274, Berlin, Heidelberg, 2002. Springer-Verlag.
- [5] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10):P10008, 2008.
- [6] Derek Bruening, Timothy Garnett, and Saman Amarasinghe. An infrastructure for adaptive dynamic optimization. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, pages 265–275, 2003.
- [7] John Bunge and M. Fitzpatrick. Estimating the number of species: A review. *Journal of the American Statistical Association*, 88(421):364–373, 1993.
- [8] Anne Chao. Nonparametric estimation of the number of classes in a population. *Scandinavian Journal of Statistics*, 11(4):265–270, 1984.

- [9] Chun-Huo Chiu, Yi-Ting Wang, Bruno A. Walther, and Anne Chao. An improved nonparametric lower bound of species richness via a modified Good–Turing frequency formula. *Biometrics*, 70(3):671–682, 2014.
- [10] Hyoun Kyu Cho, Tipp Moseley, Richard Hank, Derek Bruening, and Scott Mahlke. Instant profiling: Instrumentation sampling for profiling datacenter applications. In *Proceedings of the 2013 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 1–10, 2013.
- [11] William G. Cochran. *Sampling Techniques*. John Wiley & Sons, 3rd edition, 1977.
- [12] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.
- [13] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.
- [14] David Eklov and Erik Hagersten. StatStack: Efficient modeling of LRU caches. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 55–65, 2010.
- [15] Free Software Foundation. *The GNU C Library Reference Manual*, 2024. Section “Memory Allocation Tunables” (M_MMAP_THRESHOLD), <https://www.gnu.org/software/libc/manual/>.
- [16] Edgar Gabriel, Graham E. Fagg, George Bosilca, Thara Angskun, Jack J. Dongarra, Jeffrey M. Squyres, Vishal Sahay, Prabhanjan Kambadur, Brian Barrett, Andrew Lumsdaine, Ralph H. Castain, David J. Daniel, Richard L. Graham, and Timothy S. Woodall. Open MPI: Goals, concept, and design of a next generation MPI implementation. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface (EuroPVM/MPI)*, pages 97–104, 2004.
- [17] Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo, Ananth Kalyanaraman, and Assefaw H. Gebremedhin. miniVite: A graph analytics benchmarking tool for massively parallel systems. In *Proceedings of the IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems Workshop (PMBS)*, 2018.
- [18] I. J. Good. The population frequencies of species and the estimation of population parameters. *Biometrika*, 40(3–4):237–264, 1953.
- [19] James A. Hanley and Brenda MacGibbon. Creating non-parametric bootstrap samples using Poisson frequencies. *Computer Methods and Programs in Biomedicine*, 83(1):57–62, 2006.
- [20] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.
- [21] D. G. Horvitz and D. J. Thompson. A generalization of sampling without replacement from a finite universe. *Journal of the American Statistical Association*, 47(260):663–685, 1952.
- [22] Intel Corporation. *Pin 3.30 User Guide*, 2023. <https://software.intel.com/sites/landingpage/pintool/docs/98830/Pin/doc/html/index.html>.
- [23] R. E. Kessler, Mark D. Hill, and David A. Wood. A comparison of trace-sampling techniques for multi-megabyte caches. *IEEE Transactions on Computers*, 43(6):664–675, 1994.

- [24] Ozgur O. Kilic, Nathan R. Tallent, Yasodha Suriyakumar, Chenhao Xie, Andrés Marquez, and Stephane Eranian. Memgaze: Rapid and effective load-level memory trace analysis. In *2022 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 484–495, 2022.
- [25] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI ’05, page 190–200, New York, NY, USA, 2005. Association for Computing Machinery.
- [26] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger. Evaluation techniques for storage hierarchies. *IBM Systems Journal*, 9(2):78–117, 1970.
- [27] Samuel Miksits, Ruimin Shi, Maya Gokhale, Jacob Wahlgren, Gabin Schieffer, and Ivy Peng. Multi-level memory-centric profiling on arm processors with arm spe. In *Proceedings of the SC ’24 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, SC-W ’24, page 996–1005. IEEE Press, 2025.
- [28] Nafis Mustakin, Joseph Manzano, and Daniel Wong. MemPrint: Constructing program memory footprint estimations using statistical methods from sparsely sampled Pin-based memory traces. Manuscript, 2025.
- [29] Nicholas Nethercote and Julian Seward. Valgrind: a framework for heavyweight dynamic binary instrumentation. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI ’07, page 89–100, New York, NY, USA, 2007. Association for Computing Machinery.
- [30] Aleix Roca Nonell, Balazs Gerofi, Leonardo Bautista-Gomez, Dominique Martinet, Vicenç Beltran Querol, and Yutaka Ishikawa. On the applicability of pebs based online memory access tracking for heterogeneous memory management at scale. In *Proceedings of the Workshop on Memory Centric High Performance Computing*, MCHPC’18, page 50–57, New York, NY, USA, 2018. Association for Computing Machinery.
- [31] OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 5.1*, 2020.
- [32] SeongJae Park, Yunjae Lee, and Heon Y. Yeom. Profiling dynamic data access patterns with controlled overhead and quality. In *Proceedings of the 20th International Middleware Conference Industrial Track*, pages 1–7, 2019.
- [33] H. Patil, R. Cohn, M. Charney, R. Kapoor, A. Sun, and A. Karunanidhi. Pinpointing representative portions of large intel itanium programs with dynamic instrumentation. In *37th International Symposium on Microarchitecture (MICRO-37’04)*, pages 81–92, 2004.
- [34] Louis-Noël Pouchet. PolyBench/C 4.2.1: The polyhedral benchmark suite, 2016. <https://sourceforge.net/projects/polybench/>.
- [35] Amanda Raybuck, Tim Stamler, Wei Zhang, Mattan Erez, and Simon Peter. HeMem: Scalable tiered memory management for big data applications and real NVM. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, pages 392–407, 2021.
- [36] Julian Seward and Nicholas Nethercote. *Valgrind Massif: Heap Profiler*, 2015. Available at <http://valgrind.org/docs/manual/ms-manual.html>.

- [37] Timothy Sherwood, Erez Perelman, Greg Hamerly, and Brad Calder. Automatically characterizing large scale program behavior. In *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 45–57, 2002.
- [38] Guy L. Steele, Doug Lea, and Christine H. Flood. Fast splittable pseudorandom number generators. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 453–472, 2014.
- [39] The Linux Kernel Documentation. Examining process page tables (`/proc/pid/pagemap`), 2024. <https://www.kernel.org/doc/html/latest/admin-guide/mm/pagemap.html>.
- [40] The Linux Kernel Documentation. Soft-dirty PTEs, 2024. <https://www.kernel.org/doc/html/latest/admin-guide/mm/soft-dirty.html>.
- [41] The Linux Kernel Documentation. Transparent hugepage support, 2024. <https://www.kernel.org/doc/html/latest/admin-guide/mm/transhuge.html>.
- [42] Carl A. Waldspurger, Nohhyun Park, Alexander Garthwaite, and Irfan Ahmad. Efficient MRC construction with SHARDS. In *Proceedings of the 13th USENIX Conference on File and Storage Technologies (FAST)*, pages 95–110, 2015.
- [43] Jake Wires, Stephen Ingram, Zachary Drudi, Nicholas J. A. Harvey, and Andrew Warfield. Characterizing storage workloads with counter stacks. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 335–349, 2014.
- [44] Roland E. Wunderlich, Thomas F. Wenisch, Babak Falsafi, and James C. Hoe. SMARTS: Accelerating microarchitecture simulation via rigorous statistical sampling. In *Proceedings of the 30th Annual International Symposium on Computer Architecture (ISCA)*, pages 84–95, 2003.

A Options of the Pin tool

Table 4 lists the options of `memprint_trace`. The tool is built against the Pin kit with `make -C pintool PIN_ROOT=<path>`, and `scripts/run.sh` runs a workload natively and under the tool with these options.

B Output files

Each run writes files named `<Prefix>_<name>_<interval>_<pid>`, where the prefix is `Buffered` (splitter), `Sampled` (sampler) or `Spatial` (spatial).

- The summary file (`.csv`) and one file per bin (`_SubSample_<interval>_bin_<j>.csv`) have the columns `FunctionName`, `MemUsageObs` (footprint in bytes), `UniqueAddresses`, `CountObs` (references counted) and `SamplingInterval`, as in `MemPrint`.
- The timeline (`_timeline.csv`, with `-snapshot`) has one row per footprint per snapshot, with the columns `Time`, `SamplingInterval`, `Bin` (`-1` for the main footprint, `-2` for the union of an interval’s bins), `MemUsageObs`, `UniqueAddresses`, `CountObs`, `FreedBytes`, `Singletons`, `Doubletons`, `Tripletons`, `Quadrupletons` (f_1 to f_4) and `Discovered` (how often an address entered the sample).

Table 4: Options of `memprint_trace`.

Option	Meaning
<code>-mode</code>	<code>splitter</code> , <code>sampler</code> or <code>spatial</code> .
<code>-i</code>	Sampler: sample 1 in i references. Spatial: select 1 in i addresses (chunks in windowed mode).
<code>-s</code>	Sampler: number of bootstrap bins. Spatial: number of hash buckets.
<code>-r</code>	Sampler: replication factor, $\lambda = s/r$.
<code>-intervals</code> , <code>-bins</code>	Splitter: sampling intervals and bins per interval.
<code>-seed</code>	Seed of the random number generators and of the hash salt (default: the process id).
<code>-track_frees</code>	Remove freed ranges (malloc family and munmap) from all footprints.
<code>-snapshot</code>	Write a timeline row set every N references.
<code>-stop</code>	Write the outputs after N references and detach.
<code>-buffer_pages</code>	Trace buffer pages per thread (default 1024, or 16 with snapshots or free tracking).
<code>-window</code> , <code>-period</code>	Spatial: check references only for W of every P references (requires <code>-snapshot</code> , and turns on free tracking).
<code>-dirty_every</code>	Windowed: clear soft-dirty bits every N references (default: every ten snapshots; 0: use residency for reused blocks).
<code>-outdir</code> , <code>-name</code>	Output directory and trace name.

- The windowed file (`_windowed.csv`, with `-window`) has one row per snapshot, with the columns `Time`, `Watching` (1 inside a window), `Windows` (completed), `AllocatedBytes`, `Blocks`, `FreshBytes`, `FreshResident`, `ReusedBytes`, `ReusedResident`, `FreshEst`, `ReusedEst` and `OtherEst` (selected footprint $\times i$ in fresh blocks, in reused blocks and outside blocks), `FreshCovered` and `ReusedCovered` (bytes covered $\times i$), `Density` (of all reused blocks), `Estimate` ((5)), `Window`, `Period` and `ReusedWritten` (bytes of reused blocks on written pages).

The analysis package scores windowed runs against a splitter run in the same directory with `python -m memprint timeline windowed <workload>`, and `paper/make_figures.py` regenerates the figures of this report from the experiment outputs.